

Incoresoft VMS Plugin SDK

Overview for technical decision makers

Target release line: VMS 25.1 (Javalin 5, Java 21, Vue 3.4, Module Federation)

Generated: 2026-10-05

00 — VMS Plugin SDK at a glance

For technical decision makers evaluating a VMS integration. Two pages. The developer documentation starts in 01.

What a VMS plugin is

A plugin extends Incoresoft VMS from the inside. It is **one jar** containing a Java 21 backend extension and a Vue 3 frontend bundle. The VMS core loads it at runtime — no restart, no fork of the product — and users manage it in Admin Center or install it from the Incoresoft Marketplace.

What a plugin can do

Area	Capability
Own business logic	REST and WebSocket endpoints on the VMS server, protected by VMS authentication; own database tables in the VMS database; background processing; retention of its data through the standard storage cleaner
Own UI	A tab in the client application (any number of pages), a section in Admin Center, Video-wall cells, entries in the "+" menu and the admin sidebar, all built with the Incoresoft UI kit so they look native
Security model	Its own permissions, shown and granted in Admin Center → Roles like built-in ones, enforced by the server and respected by the UI
Events & Rules	New rule types for the alarm engine: users configure "when X happens, notify / record / trigger output", the plugin raises alarms and notifications with its own payload and its own alarm details UI
Maps & plans	Own object types placed on maps and floor plans (doors, terminals, sensors) with live status and event pop-ups
Search	Own result type in the global Search page with filters, cards and details
Platform data	Read access to cameras, streams, recording servers, users and roles; snapshots and video around an event

A plugin with your own video analytics is an ordinary plugin: your analytics server posts its detections to the plugin, the plugin stores them with the camera they came from, and VMS adds alarms with video, notifications and search. No Incoresoft analytics server is involved.

What a partner needs

Skill s	Java 21, Maven, Guice, Javalin, SQL (jOOQ/Liquibase); Vue 3, TypeScript, Webpack Module Federation
Tooling	JDK 21, Maven 3.9, Node.js 18+, a VMS 25.1 installation for development (evaluation licence from Incoresoft)
Libraries	The public Java host API (<code>com.incoresoft.commons:middleware</code> , plus <code>com.incoresoft:device-driver-sdk</code> for plugins that resolve cameras/devices) on Maven Central, and the UI packages <code>@incoresoft/incoresoft-ui</code> , <code>-icons</code> and <code>-toasts</code> on the public npm registry; nothing to configure on either side
SDK package	This documentation, a complete sample plugin as two templates (backend and frontend)

How it goes from idea to customers

1. **Kick-off** — partner receives credentials, the SDK package and a development VMS.
2. **Scaffold** — copy the two templates, rename, run them against the development VMS (first visible result in under an hour).
3. **Develop** — build the jar and upload it to the development VMS in Admin Center; the new version replaces the old one without restarting VMS and keeps the plugin's data and rules. The host finds the plugin's UI automatically.
4. **Package** — one `mvn package` produces the jar with the frontend inside.
5. **Deliver** — hand the jar to your customers: they install and update it in Admin Center (Settings → Marketplace → Extra Settings → Install Plugin from Disk) or over the API. A listing in the Incoresoft Marketplace, with one-click install and updates, is arranged with Incoresoft.

Rules of the game

- **One plugin version series per VMS release line.** VMS 25.1 accepts plugins versioned `25.1.x`; a new VMS line (e.g. 26.1) needs a rebuilt, re-versioned plugin. The SDK libraries follow the same lines and their API is kept binary compatible within a line.
- **Plugins run inside the VMS process with full trust.** That is what makes them powerful; it is also why partners are expected to follow the SDK's route, permission and resource conventions.
- **Identifiers are yours to choose once.** Plugin id, module name and permission ids are part of the customer's data after the first installation and must stay stable.
- **Languages.** The UI framework supports English, Ukrainian and Spanish; plugins ship their own translations.

Where to read next

`01 Architecture` for the technical picture, `02 Getting started` for the hands-on walkthrough, `04 Frontend` §10 for the complete list of UI extension points.